

Weekly Report #4

Benchmarking Stream Clustering Algorithms in River

Contributor	Nguyen Van A	Reporting period	22 Jun 2026 – 28 Jun 2026
Mentors	Dr. B, Dr. C	Phase	Phase 2 - HPO methodology
Repository	github.com/user/repo	Report date	September 19, 2026

1. Executive Summary

Objective - Turn the HPO methodology from weeks 2–3 into a working pipeline for CluStream and DenStream. Each tuning method picks *one* hyperparameter set per algorithm, and we run that set on full-length datasets to see whether it beats River’s defaults.

Status - **IN PROGRESS** On track

- **DenStream:** tuning helps. P2R is the only method that beats the default on all three datasets, but it is about 220× slower on Coverttype.
- **CluStream:** ARI stays below 0.025 for every method, so ARI cannot guide its tuning. Tuned sets are only faster.
- **Main issue:** the average ARI is driven by KDDCup99, and Coverttype cannot be tuned at all. All results so far are in-sample with 2 seeds.

Progress tracker

Table 1: Weekly plan versus reality, week 4.

Weekly plan item	Phase ref.	Status	Completion
Define HPO protocols and search grids	P2	DONE	100%
Stage A: tune on 5 datasets, 50k samples each	P2	DONE	100%
Stage B: run chosen sets on 3 full-length datasets	P2	DONE	100%
Out-of-sample evaluation on 6 unseen datasets	P2	IN PROGRESS	50%
Headroom-based selection and fidelity check	P2	PLANNED	0%

Takeaway. The pipeline runs end to end. The out-of-sample test is coded and is the priority for next week.

Overall: Week 4/18 - on schedule.

2. Completed Tasks

C1. HPO protocols and their theory P2

We run every candidate configuration once and store the scores in one matrix, then read it in different ways [3]. Let $\bar{M}[\lambda, d]$ be the ARI of configuration λ on dataset d , averaged over seeds. The protocols differ only in what they may see when choosing λ .

- **Default:** River’s constructor defaults. No selection.
- **Arm 1:** the set with the best average ARI over all datasets, $\lambda^* = \arg \max_{\lambda} \frac{1}{|D|} \sum_d \bar{M}[\lambda, d]$.
- **Arm 2, leave-one-dataset-out (LODO)** [1]. For each held-out dataset d , choose on the

others and score on d :

$$\lambda_{-d}^* = \arg \max_{\lambda} \frac{1}{|D| - 1} \sum_{d' \neq d} \bar{M}[\lambda, d'], \quad \text{score}_{\text{LODO}}(d) = \bar{M}[\lambda_{-d}^*, d].$$

The final set is the one chosen by the most rounds. The gap between the in-sample score and the LODO score measures how optimistic a method is.

- **Arm 3, relative parameters.** We do not scale the data, so a fixed ε means different things on different datasets. Following [2], we tune a unit-free factor c and set

$$\varepsilon_d = c \cdot s(D_d), \quad q = \text{ratio} \cdot k,$$

where $s(D)$ is the median pairwise distance on the first 5,000 samples. For CluStream, q/k is the micro-ratio studied in [4]. Arm 3 applies LODO to this relative grid.

- **P2R, LODO on a selection rule.** Every clustering algorithm has one knob that sets how many clusters it finds: k for CluStream, ε for DenStream. We call it the granularity g . The right g depends on the data, so a value tuned on other datasets rarely fits a new one. P2R therefore handles the two kinds of parameters differently:
 - The other parameters θ , such as β and μ , are chosen by LODO as usual.
 - g is not carried over. On each new dataset we run every level of g , score each run with a criterion that needs no labels, and pick a level from the resulting curve:

$$\hat{g}(d) = R(\{(g, \ell_g(d))\}_g).$$

In short, LODO hands over a value, while P2R hands over a way of choosing the value. For DenStream, the same rule picked $c = 0.8$ on KDDCup99, 0.2 on Covertypes and 0.4 on INSECTS.

- **Label-free criterion** [2]. We use the trace criterion with $\hat{S}_{ij} = -\|y_i - y_j\|^2$, which reduces to

$$\ell = -2 \sum_k \text{WCSS}_k, \quad \text{WCSS}_k = SS_k - \frac{\|LS_k\|^2}{n_k},$$

so it can be updated online from the cluster features (n_k, LS_k, SS_k) .

- **Knee rule.** ℓ_g keeps rising as clusters get finer, so taking its maximum always picks the finest level. We normalise the curve to $x_g, y_g \in [0, 1]$ and pick the point farthest from the line joining its two ends:

$$\hat{g} = \arg \max_g |(y_G - y_1) x_g - y_g + y_1|.$$

C2. Experiment setup

P2

- **Data:** raw data, shuffled, then streamed. No scaler.
- **Seeds:** sets are chosen on seed 1 and all reported scores average seeds 2 and 3, so a set cannot win by luck on one seed.
- **Grids:** every value comes from the original papers [4, 5]. DenStream requires $\beta\mu > 1$ and CluStream requires $q \geq k$.
 - CluStream: $k \in \{2, \dots, 64\}$, $q \in \{50, 100, 200\}$ or $\text{ratio} \in \{2, 5, 10\}$, $t \in \{1, 2, 4\}$.
 - DenStream: $\varepsilon \in \{0.02, \dots, 16\}$ or $c \in \{0.05, \dots, 3.2\}$, $\mu \in \{2, 5, 10, 20\}$, $\beta \in \{0.2, 0.5, 0.75\}$.
- **Stage A:** run all ≈ 230 configurations on 50k samples of 5 real datasets and let each

method pick one set.

- **Stage B:** run the chosen sets on full KDDCup99 10%, Covertypes and INSECTS. All 46 runs finished.

3. Results & Visualization

3.1 Stage A: chosen sets

Table 2: Chosen sets. Absolute values are listed as KDDCup99 / Covertypes / INSECTS.

Algorithm	Method	Chosen set
DenStream	Default	$\varepsilon = 0.02, \beta = 0.75, \mu = 2$
	Arm 1 = Arm 2	$\varepsilon = 5.5, \beta = 0.2, \mu = 20$
	Arm 3	$\varepsilon = 1.6 \cdot s(D) = 1360/3668/5221, \beta = 0.5, \mu = 5$
	P2R	$\varepsilon = 680/459/1305, \beta = 0.5, \mu = 5$
CluStream	Default	$k = 5, q = 100, t = 2$
	Arm 1 = Arm 2	$k = 8, q = 50, t = 2$
	Arm 3	$k = 8, q = 16, t = 2$
	P2R	$k = 4/8/16, q = 8/16/32, t = 2$

Table 3: Stage A ARI, 50k samples, averaged over 5 datasets.

Algorithm	Method	In-sample	LODO	Optimism
DenStream	Default	0.044	–	–
	Arm 2	0.073	0.044	0.029
	Arm 3	0.113	0.079	0.034
	P2R	0.110	0.109	0.001
CluStream	all methods	≤ 0.005	≤ 0.004	–

Takeaway. For DenStream, relative parameters help: Arm 3 beats Arm 2 by 0.035 on LODO. P2R loses almost nothing on unseen datasets because it re-reads the granularity on each one.

3.2 Stage B: full-length runs

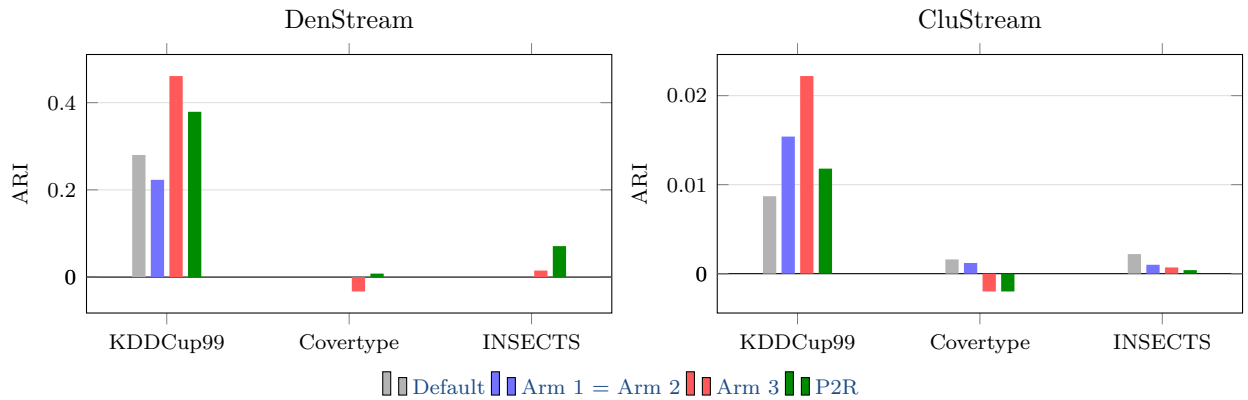


Figure 1: Full-stream ARI, mean of seeds 2 and 3. The two panels use different y scales.

Table 4: DenStream on full-length datasets.

Method	ARI			Throughput (samples/s)		
	KDD	Cov	INS	KDD	Cov	INS
Default	0.280	0.000	0.000	9,926	71,323	29,880
Arm 1 = Arm 2	0.223	0.000	0.000	6,391	70,119	30,407
Arm 3	0.461	-0.033	0.015	6,778	25,479	27,148
P2R	0.379	0.008	0.071	3,952	328	384

Takeaway. The default and Arms 1–2 collapse to one cluster on Covertypes and INSECTS because ϵ is far too small for their scale. Arm 3 is best on KDDCup99 but negative on Covertypes. P2R is the only method that beats the default everywhere, at the cost of many more micro-clusters and much lower throughput. For CluStream, all differences are within seed noise; tuned sets only run 3–14× faster.

4. Issues

The average ARI is mostly KDDCup99. When we average ARI over datasets, Arm 3 and P2R look clearly better than the default. But nearly all of that gain comes from KDDCup99. On Covertypes and INSECTS both methods stay close to zero. This matters because KDDCup99 is not a fair test: without scaling, a single feature holds 99.8% of its variance, so doing well on it mostly means cutting one axis at the right place. We plan to score each dataset against its own best possible result instead of using raw ARI (Section 5). We would like the mentors’ opinion on whether this is a reasonable way to report the results.

Some datasets cannot be tuned at all. Even when we pick the best configuration for Covertypes using its labels, ARI only reaches 0.018. CluStream is worse: its best ARI on any dataset is 0.032. So the problem is not the tuning method; nothing in the search space works on these cases. Keeping them in the selection step only adds noise. We will mark them as untunable and leave them out of selection, while keeping the decision not to scale the data.

The full-length results are still in-sample. The three datasets in Stage B were also used to choose the sets, and we only have two seeds, so the numbers are optimistic. We also have not checked that 50k samples rank configurations the same way full streams do; so far this holds for the five chosen sets, which is too few to be sure. The out-of-sample test and a fidelity check should settle both points.

We chose which parameters to tune by argument, not by measurement. DenStream has six parameters. We tuned ϵ , β and μ and fixed the other three because the original paper suggests they matter little [5]. We never checked this on our own data. Our Stage A grid does let us check part of it, since it covers every combination of ϵ , β and μ . Table 5 splits the ARI variance by parameter.

Table 5: Share of DenStream ARI variance explained by each parameter, 50k samples, seed 1.

Dataset	ϵ	β	μ	Interaction
Shuttle	90%	0%	2%	9%
INSECTS	74%	0%	3%	23%
Keystroke	71%	2%	11%	18%
KDDCup99	18%	0%	13%	64%

Two things stand out. First, β has almost no effect, so a third of our tuning budget went to a parameter that does not matter. Second, on KDDCup99 most of the variance comes from the interaction: the best μ changes with ε . P2R assumes the opposite, since it fixes μ first and picks ε afterwards. We still know nothing about the three fixed parameters because they were never varied. Covertypes is left out because its ARI barely moves.

P2R is expensive. P2R picks a smaller ε and keeps many more micro-clusters, which makes it about $220\times$ slower than the default on Covertypes. Arm 3 is much cheaper but fails on Covertypes. We will report both as two options on the quality–cost trade-off, and we would like guidance on how much extra cost is acceptable for a recommended set.

5. Proposed Solutions & Next Week’s Plan

(a) **Headroom.** Measure each method against the best score reachable on that dataset:

$$h(d) = \frac{\text{ARI}_{\text{method}}(d) - \text{ARI}_{\text{default}}(d)}{\text{ARI}_{\text{best}}(d) - \text{ARI}_{\text{default}}(d)}.$$

$h = 1$ means as good as the best, $h = 0$ means no better than the default. A dataset is untunable if $\text{ARI}_{\text{best}} < 0.05$ or $\text{ARI}_{\text{best}} - \text{ARI}_{\text{default}} < 0.02$. These thresholds are our own choice.

- (b) **Select the way we read.** Use headroom instead of raw ARI when choosing sets, and drop untunable datasets from selection. This reuses the existing 50k runs.
- (c) **Fidelity check** [6]. Rerun the top configurations at full length and compute the Spearman rank correlation with their 50k scores. If it is low, tune on longer streams.
- (d) **Check which parameters matter before tuning** [7, 8]. Try about 100 random settings of all six DenStream parameters on each tuning dataset, then measure how much each parameter, and each pair, changes ARI. Only the parameters that clearly matter go into the grid; the rest stay at their defaults. If μ and ε still interact strongly, P2R should pick them together on each dataset instead of fixing μ in advance.
- (e) **Out-of-sample test.** Run the chosen sets on 6 unseen datasets with 4 seeds, and compare each method with the default using win/tie/loss counts and a Wilcoxon test.

Table 6: Plan for week 5.

#	Task	Definition of done (verifiable)	Phase
N1	Add headroom and win/tie/loss to the report	Table printed for all tuning datasets	P2
N2	Headroom-based selection	New chosen sets computed from existing runs	P2
N3	Fidelity check, 50k vs full length	Spearman correlation per dataset	P2
N4	Out-of-sample evaluation	Results on 6 unseen datasets with Wilcoxon test	P2
N5	Parameter screening for DenStream	Importance of all six parameters on each tunable dataset	P2

Takeaway. N1 and N2 need no new runs. N4 decides the outcome: if P2R does not beat the default on most unseen datasets, we will report that ARI-based HPO does not transfer for these algorithms.

6. References / Links

Code produced this week

Ref	Description	Location
<code>hpo_compare.py</code>	Stage A: grids, tuning, set selection	<code>hpo_pipeline/</code>
<code>rules.py</code>	P2R selection rules	<code>hpo_pipeline/</code>
<code>final_eval.py</code>	Stage B: full-length runs	<code>hpo_pipeline/</code>
Notebook	Evaluation of the chosen sets	<code>result/hpo_final_sets_evaluation.ipynb</code>

Literature and documentation

References

- [1] V. Cerqueira, H. M. Gomes, M. Heyden, B. Pfahringer, and A. Bifet, “A Framework for Evaluating and Benchmarking Concept Drift Detection Methods,” *KDD*, 2026. arXiv:[2606.07789](#).
- [2] X. Fan, Y. Yue, P. Sarkar, and Y. X. R. Wang, “On Hyperparameter Tuning in General Clustering Problems,” *ICML*, 2020.
- [3] L. Franceschi et al., “Hyperparameter Optimization in Machine Learning,” arXiv:[2410.22854](#), 2024.
- [4] C. C. Aggarwal, J. Han, J. Wang, and P. S. Yu, “A Framework for Clustering Evolving Data Streams,” *VLDB*, 2003.
- [5] F. Cao, M. Ester, W. Qian, and A. Zhou, “Density-Based Clustering over an Evolving Data Stream with Noise,” *SDM*, 2006.
- [6] A. Klein, S. Falkner, S. Bartels, P. Hennig, and F. Hutter, “Fast Bayesian Optimization of Machine Learning Hyperparameters on Large Datasets,” *AISTATS*, 2017.
- [7] F. Hutter, H. Hoos, and K. Leyton-Brown, “An Efficient Approach for Assessing Hyperparameter Importance,” *ICML*, 2014.
- [8] J. N. van Rijn and F. Hutter, “Hyperparameter Importance Across Datasets,” *KDD*, 2018.